

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА №42

КУРСОВАЯ РАБОТА (ПРОЕКТ)
ЗАЩИЩЕНА С ОЦЕНКОЙ
РУКОВОДИТЕЛЬ

доцент, канд. техн. наук
должность, уч. степень, звание

подпись, дата

О. И. Красильникова
инициалы, фамилия

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
К КУРСОВОЙ РАБОТЕ (ПРОЕКТУ)

по дисциплине:

Web-технологии

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № _____ 4321

подпись, дата

Буренков Г. В.
инициалы, фамилия

Санкт-Петербург 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	2
1 Дизайн сайта	3
1.1 Макет сайта	3
1.2 Применение технологии каскадных таблиц стилей.....	5
1.3 Графическое оформление web-страницы.....	9
1.4 Создание навигационных панелей для сайта.....	10
2 Web-сценарии сайта на языке JavaScript	12
2.1 Назначение языка JavaScript.....	12
2.2 Разработка JS-скриптов для сайта.....	12
3 Валидация кода и продвижение сайта	15
3.1 Валидация кода сайта	15
3.2 Способы продвижения сайта	16
3.3 Используемые приемы продвижения сайта	16
ЗАКЛЮЧЕНИЕ	18
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	19
ПРИЛОЖЕНИЕ	20

ВВЕДЕНИЕ

В современных условиях качественный веб-сайт является обязательным требованием для успеха в строительной отрасли. Веб-ресурс позволяет компании «Евклид» эффективно демонстрировать портфолио, описывать услуги и автоматизировать сбор заявок от клиентов. Использование архитектуры одностраничных приложений (SPA) на базе React обеспечивает высокую скорость работы и плавность интерфейса, что укрепляет имидж бренда и повышает удобство для пользователей.

Особое внимание при разработке уделяется созданию интуитивно понятного интерфейса, который способен удерживать внимание посетителя и эффективно направлять его к целевому действию. Интеграция современных фронтенд-инструментов позволяет реализовать сложные визуальные переходы и мгновенную реакцию на действия пользователя, что в долгосрочной перспективе способствует росту лояльности к бренду. Такой подход превращает сайт из пассивного источника информации в активный канал продаж и профессиональной коммуникации с заказчиком.

Целью данной работы является проектирование и разработка адаптивного корпоративного сайта строительной компании с использованием современных технологий. В ходе работы решаются задачи верстки интерфейса, создания интерактивных сценариев на JavaScript и проведения поисковой оптимизации. Итоговый продукт представляет собой функциональную платформу, включающую описание услуг компании, этапов работ, ответы на вопросы и формы обратной связи, соответствующие техническим и правовым нормам ведения бизнеса.

1 Дизайн сайта

Процесс визуального и структурного проектирования корпоративного сайта «Евклид» является важнейшим этапом разработки, направленным на формирование единой концепции взаимодействия пользователя с брендом. Дизайн сайта объединяет в себе не только эстетические решения, но и глубокую проработку информационной архитектуры, основанной на модели одностраничного приложения (SPA). Такой подход позволяет создать интуитивно понятную среду, где каждый графический элемент и блок контента логически связаны между собой, обеспечивая бесшовный переход от стадии знакомства с услугами до формирования заявки. Основная задача дизайна в данном проекте заключается в трансляции ценностей строительной компании — надежности, профессионализма и технологичности — через выверенные визуальные образы и современные механизмы отображения информации.

1.1 Макет сайта

Визуальная концепция проекта была детально проработана в графическом редакторе Figma, что позволило на этапе проектирования согласовать все цветовые акценты и типографические решения. Общий вид макета главной страницы сайта представлен на рисунке 1. Макет ориентирован на создание атмосферы надежности и масштабности, характерной для строительного бизнеса. Основная верстка сайта реализована с использованием компонентного подхода, где каждый логический блок выделен в отдельный программный модуль. Для формирования сетки и позиционирования элементов применены технологии CSS Flexbox, которые обеспечивают высокую гибкость интерфейса. Адаптивность макета является приоритетной задачей, поэтому в коде предусмотрены медиа-запросы, изменяющие структуру отображения для различных устройств. На рисунке 2 наглядно демонстрируется адаптивная версия сайта для мобильных устройств, где элементы перестраиваются в вертикальные блоки для удобства прокрутки

одним пальцем. Использование инструмента автоматизации сборки Vite позволяет оптимизировать итоговый код, уменьшая время первой отрисовки контента и обеспечивая корректное отображение на всех современных платформах.

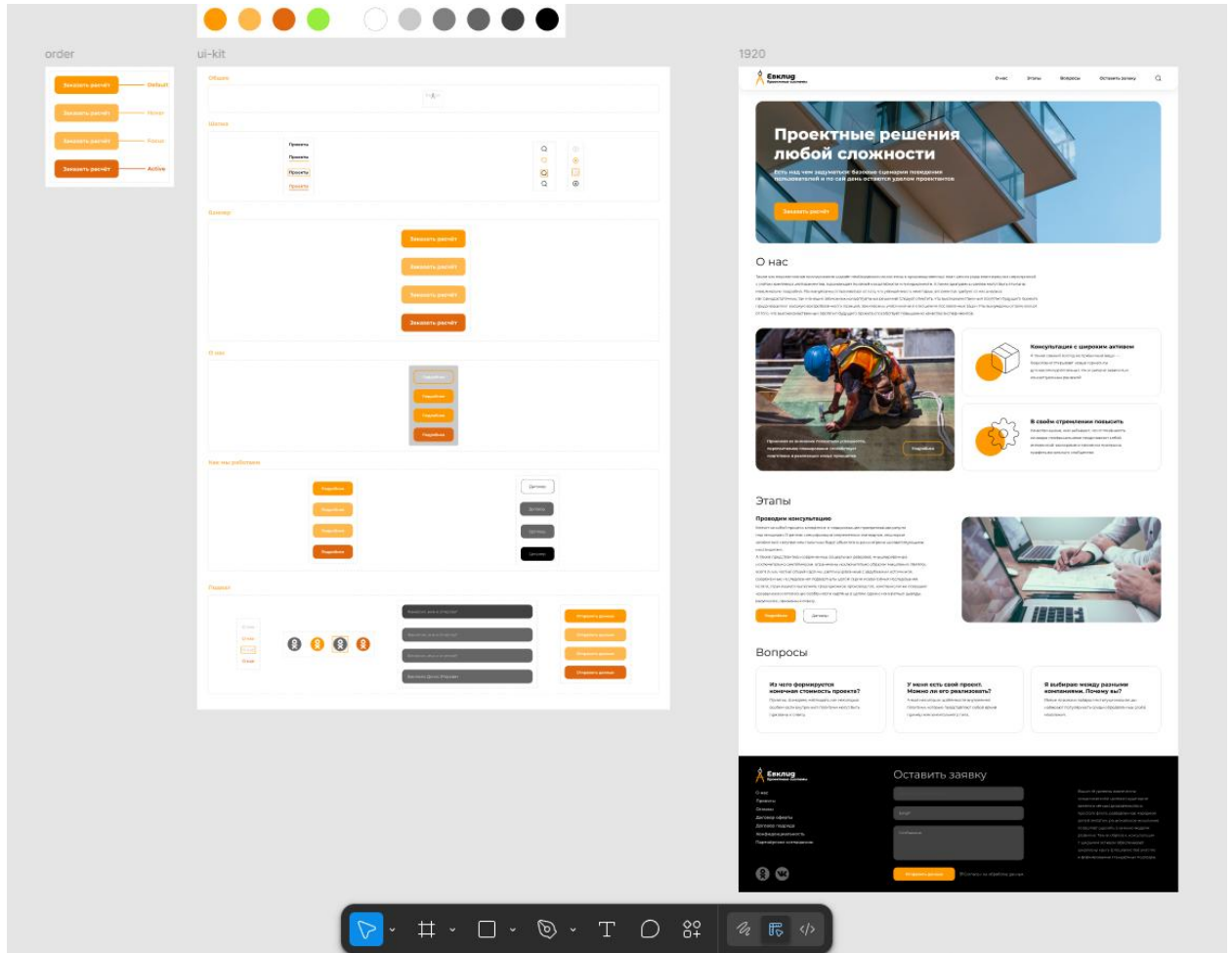


Рисунок 1 — Дизайн-макет сайта в Figma

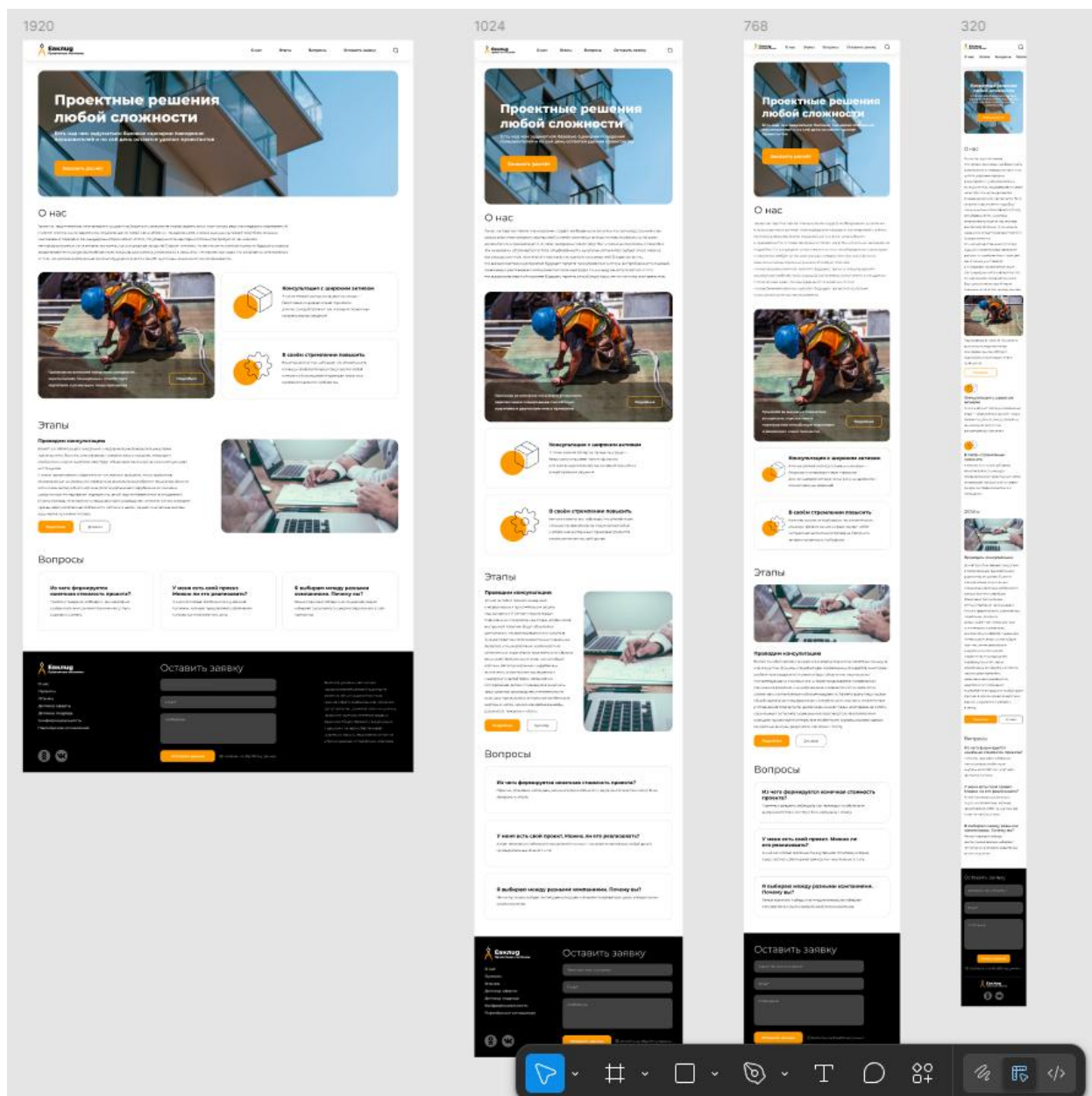


Рисунок 2 — Адаптивная версия сайта для мобильных устройств

1.2 Применение технологии каскадных таблиц стилей

Техническое оформление внешнего вида сайта реализовано через систему каскадных таблиц стилей в формате чистого CSS3, что обеспечивает максимальную производительность программного продукта. Для управления глобальными параметрами дизайна, такими как цветовая палитра, шрифтовые наборы и стандартные отступы, активно используются CSS-переменные (Custom Properties), определенные в корневом селекторе `:root`. Фрагмент кода с объявлением системных переменных представлен на рисунке 3. Например,

для главного акцентного цвета используется переменная `--orange`, что позволяет мгновенно изменять цветовую схему всего приложения путем корректировки всего одной строки кода. Это обеспечивает высокую степень консистентности оформления и значительно упрощает долгосрочную поддержку проекта. Архитектура стилей выстроена в строгом соответствии с методологией БЭМ (Блок-Элемент-Модификатор), которая позволяет избежать конфликтов имен и делает код предсказуемым. Логика именования классов по методологии БЭМ на примере шапки сайта продемонстрирована на рисунке 4. Использование селекторов вида `.header__nav-list-item` гарантирует, что стилизация одного компонента не повлияет на другие части интерфейса, обеспечивая модульность и чистоту структуры.

Особое внимание в проекте уделено вопросам обеспечения кроссбраузерности, что критично для корпоративного ресурса с широкой целевой аудиторией. Для унификации отображения элементов во всех современных браузерах (Chrome, Firefox, Safari, Edge) в проект интегрирован файл `normalize.css`. Эта библиотека исправляет базовые различия в рендеринге стандартных HTML-тегов, такие как особенности отображения полей ввода в iOS или специфические отступы списков в старых версиях Internet Explorer. Пример кода нормализации стилей для обеспечения кроссбраузерной стабильности представлен на рисунке 5. Дополнительно, инструменты сборки Vite в связке с PostCSS автоматически добавляют необходимые вендорные префиксы (`-webkit-`, `-moz-`), гарантируя корректную работу Flexbox-контейнеров и CSS-анимаций даже на устаревших версиях браузерных движков. Это позволяет минимизировать визуальные расхождения и обеспечить стабильную работу сайта на любой пользовательской платформе.

```
61  √ :root {  
62      --light-orange: #FFB84D;  
63      --orange: #FF9900;  
64      --dark-orange: #E1670E;  
65      --white: #fff;  
66  
67  }
```

Рисунок 3 — Использование CSS-переменных (src/css/style.css, строки 61-67)

```
134 /* header */
135
136 .header {
137     margin-bottom: 50px;
138     padding: 14px 0;
139     box-shadow: 4px 4px 8px 0px rgba(34, 60, 80, 0.2);
140     border-radius: 0 0 32px 32px;
141 }
142
143 .header_shape {
144     display: flex;
145     align-items: center;
146     justify-content: space-between;
147     flex-wrap: wrap;
148 }
149
150 .header_image {
151     display: flex;
152     margin-right: auto;
153     flex-shrink: 0;
154 }
155
156 .header_image:focus-visible {
157     outline: 2px solid var(--orange);
158     outline-offset: 5px;
159 }
```

Рисунок 4 — Реализация методологии БЭМ в стилях проекта (src/css/style.css, строки 136-160)

```
11 html {
12     line-height: 1.15; /* 1 */
13     -webkit-text-size-adjust: 100%; /* 2 */
14 }
15
16 /* Sections
17 | ===== */
18
19 /**
20  * Remove the margin in all browsers.
21  */
22
23 body {
24     margin: 0;
25 }
26
27 /**
28  * Render the `main` element consistently in IE.
29  */
30
31 main {
32     display: block;
33 }
34
35 /**
36  * Correct the font size and margin on `h1` elements within `section` and
37  * `article` contexts in Chrome, Firefox, and Safari.
38  */
39
```

Рисунок 5 — Средства обеспечения кроссбраузерности (src/css/normalize.css, строки 11-33)

Реализация адаптивной верстки является ключевым фактором доступности сайта для мобильных пользователей. Стратегия адаптивности строится на использовании медиа-запросов (@media), которые поэтапно перестраивают макет при достижении контрольных точек (breakpoints: 1440px, 1024px, 768px, 320px). При уменьшении ширины экрана сложные

многоколоночные сетки трансформируются в вертикальные блоки с использованием свойства `flex-direction: column`. На рисунке 6 представлен фрагмент кода из файла `media.css`, где описывается перестроение карточек раздела «О нас» для планшетных версий. Такой подход в сочетании с относительными единицами измерения (`%`, `vw`, `calc`) позволяет интерфейсу «резиново» подстраиваться под любые разрешения экранов, сохраняя при этом читаемость текстов и удобство клика по элементам управления.

```
1  @media (max-width: 1440px) {
2
3      /* about */
4
5      .about_card-left {
6          margin-bottom: 32px;
7          width: 100%;
8          min-height: 640px;
9      }
10
11     .about_card-block-right {
12         max-width: 100%;
13     }
14
15     .about_card-list {
16         align-items: center;
17         flex-direction: column;
18         gap: 0;
19     }
20
21     .about_card-left {
22         margin-bottom: 32px;
23         max-width: 1440px;
24         gap: 10px;
25         min-height: 640px;
26         width: 100%;
27     }
28
29     .about_card-block-right {
30         max-width: 1440px;
31     }
32 }
```

Рисунок 6 — Реализация адаптивности через медиа-запросы
(`src/css/media.css`, строки 1-19)

Обеспечение интерактивности средствами CSS3 позволило создать «живой» отклик интерфейса без привлечения тяжелых JavaScript-библиотек. Для всех интерактивных элементов (кнопок, ссылок, карточек) прописаны состояния `:hover`, `:focus-visible` и `:active`. Плавность визуальных изменений достигается за счет универсального свойства `transition`, которое анимирует изменение цвета фона или трансформацию элементов в течение 0.3 секунд. Пример реализации интерактивных состояний кнопок с плавными переходами продемонстрирован на рисунке 7. Использование псевдоэлементов `::after` для создания динамических подчеркиваний в навигации и эффектов масштабирования изображений при наведении позволяет сделать

взаимодействие с сайтом «Евклид» максимально эстетичным и интуитивно понятным, повышая общую вовлеченность пользователей.

```
97  ✓ .button {  
98      color: var(--white);  
99      background-color: var(--orange);  
100     border: none;  
101     border-radius: 16px;  
102     font-weight: 700;  
103     outline: none;  
104 }  
105  
106  ✓ .button:focus-visible {  
107     background-color: var(--light-orange);  
108     border-color: var(--light-orange);  
109 }  
110  
111  ✓ .button:hover {  
112     background-color: var(--light-orange);  
113     border-color: var(--light-orange);  
114 }  
115  
116  ✓ .button:active {  
117     background-color: var(--dark-orange);  
118     border-color: var(--dark-orange);  
119 }
```

Рисунок 7 — Оформление интерактивных состояний элементов
(src/css/style.css, строки 97-119).

1.3 Графическое оформление web-страницы

Графическая система сайта «Евклид» представляет собой сбалансированное сочетание векторной и растровой графики, каждая из которых решает свои специфические задачи. Для элементов интерфейса, таких как логотипы, иконки услуг и социальные кнопки, выбран формат SVG. Использование векторного формата гарантирует идеальную четкость изображений на экранах с высокой плотностью пикселей и позволяет минимизировать объем передаваемых данных.

Растровая графика представлена высококачественными фотографиями строительных объектов и фоновыми изображениями, которые задают тематический контекст. Все фотографии прошли предварительную подготовку и оптимизацию, включая конвертацию в современный формат WebP, что позволило существенно сократить время загрузки ресурса при

сохранении высокого визуального качества. Структура хранения графических ресурсов проекта в директории `img` представлена на рисунке 8. Графическое оформление выполняет не только декоративную, но и навигационную функцию, помогая визуально разделить контент на смысловые блоки и акцентировать внимание пользователя на ключевых призывах к действию.

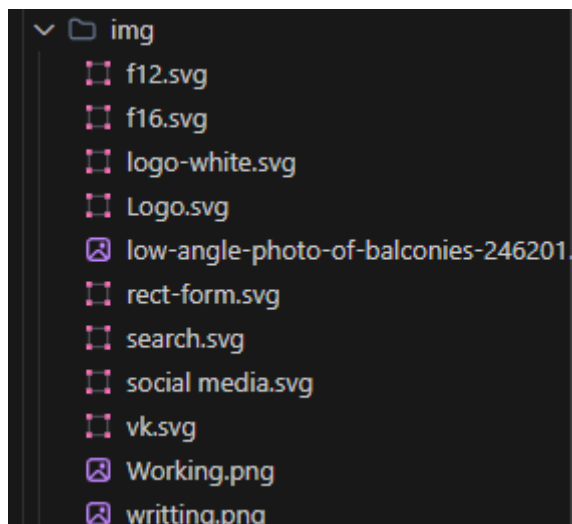


Рисунок 8 — Структура графических ресурсов проекта.

1.4 Создание навигационных панелей для сайта

Навигационный комплекс сайта спроектирован как центральный элемент управления пользовательским опытом, реализованный в отдельном компоненте `Header.jsx`. Панель навигации включает в себя набор ссылок, связанных уникальными идентификаторами разделов страницы. Техническая реализация предусматривает обработку событий клика через JavaScript, что позволяет подменять стандартное мгновенное перемещение браузера на плавный эффект скроллинга. Общий вид навигационной панели в десктопной версии представлен на рисунке 9.



Рисунок 9 — Навигационная панель сайта

В мобильной версии навигационная панель трансформируется в компактный вид для экономии экранного пространства, сохраняя при этом

полную функциональность. Логика навигации также включает в себя механизмы отслеживания активной позиции пользователя на странице для визуального выделения соответствующих пунктов меню. Это создает ощущение непрерывной обратной связи от системы и помогает пользователю легко ориентироваться в структуре одностраничного приложения, обеспечивая быстрый доступ к любой информации — от описания этапов работ до формы заказа.

2 Web-сценарии сайта на языке JavaScript

2.1 Назначение языка JavaScript

Основным инструментом для обеспечения динамического поведения и интерактивности сайта компании «Евклид» выступает язык программирования JavaScript. Применение данного языка в рамках библиотеки React позволяет перенести значительную часть вычислительной логики на сторону клиента, что минимизирует количество запросов к серверу и ускоряет отклик интерфейса. JavaScript в проекте решает задачи управления состоянием компонентов, обработки пользовательских событий и реализации сложной анимации. Программная реализация ориентирована на создание «живой» среды, где действия пользователя инициируют мгновенные визуальные изменения, такие как открытие поисковых строк, прокрутка контента или валидация вводимых данных в режиме реального времени.

2.2 Разработка JS-скриптов для сайта

Программная логика интерактивных элементов проекта распределена по специализированным функциональным компонентам, каждый из которых инкапсулирует в себе определенный сценарий поведения. Центральным узлом управления навигацией является модуль `Header.jsx`, в котором реализован алгоритм плавного скроллинга. Технически это обеспечивается через функцию перехвата стандартного события клика по ссылке и последующего вызова метода `scrollIntoView` с параметром `behavior set to smooth`. Это создает эффект профессионального и дорогого интерфейса, характерного для крупных строительных холдингов. Кроме того, в этом же компоненте интегрирована интеллектуальная система поиска. Принцип работы поискового алгоритма и логика перенаправления пользователя на основе карты ключевых слов представлены на рисунке 10. Данный сценарий анализирует вводимые пользователем данные и сопоставляет их с заранее определенной картой ключевых слов. В случае совпадения запроса с одной из категорий услуг или информационных блоков, система автоматически инициирует переход к

соответствующему разделу, что значительно сокращает время поиска информации для потенциального клиента.

```
const handleSearch = (e) => {
  e.preventDefault();
  const query = searchQuery.toLowerCase().trim();

  const keywords = {
    'нас': 'about',
    'компаний': 'about',
    'кто мы': 'about',
    'этапы': 'stages',
    'как мы работаем': 'stages',
    'вопросы': 'faq',
    'ответы': 'faq',
    'чаво': 'faq',
    'заявка': 'footer',
    'оставить заявку': 'footer',
    'контакты': 'footer',
    'связь': 'footer'
  };

  const targetId = keywords[query];
  if (targetId) {
    const elem = document.getElementById(targetId);
    if (elem) {
      elem.scrollIntoView({ behavior: 'smooth' });
      setIsSearchOpen(false);
      setSearchQuery('');
      return;
    }
  }

  // If no exact keyword match, try fuzzy matching
  const foundKeyword = Object.keys(keywords).find(k => query.includes(k) || k.includes(query));
  if (foundKeyword) {
    const elem = document.getElementById(keywords[foundKeyword]);
    if (elem) {
      elem.scrollIntoView({ behavior: 'smooth' });
      setIsSearchOpen(false);
      setSearchQuery('');
    }
  }
}
```

Рисунок 10 — Алгоритм работы поисковой системы
(src/components/Header.jsx, строки 15-55)

Другим важным техническим решением является компонент `ScrollToTop.jsx`, отвечающий за логику работы кнопки быстрого возврата в начало страницы. Работа данного скрипта базируется на постоянном мониторинге вертикального смещения окна браузера через обработчик события `scroll`. Условием появления элемента является преодоление порога в 300 пикселей от верхней границы экрана. Однако, в отличие от стандартных решений, наш сценарий включает в себя механизм интеллектуального скрывтия: если после появления кнопки пользователь перестает совершать действия по прокрутке в течение 5 секунд, срабатывает таймер, который плавно уводит кнопку в прозрачное состояние. Программная реализация логики таймера и управления видимостью кнопки показана на рисунке 11. Это

позволяет сохранять чистоту интерфейса и не отвлекать посетителя от изучения контента, когда вспомогательный инструмент ему не требуется. Логика полностью реализована через хуки состояния `useState` и эффекты `useEffect`, что обеспечивает надежную очистку ресурсов при уничтожении компонента.

```
9      const handleScroll = () => {
10        if (window.pageYOffset > 300) {
11          setIsVisible(true);
12        }
13        Ctrl+L for Agent
14        if (timeoutId) clearTimeout(timeoutId);
15        timeoutId = setTimeout(() => {
16          setIsVisible(false);
17        }, 5000);
18      } else {
19        setIsVisible(false);
20      }
21    };

```

Рисунок 11 — Логика таймера и видимости кнопки ScrollToTop
(src/components/ScrollToTop.jsx, строки 9-21)

Сценарии взаимодействия в подвале сайта, реализованные в компоненте `Footer.jsx`, направлены на обеспечение безопасности и корректности сбора данных. Функция обработки формы `handleSubmit` выполняет программную проверку состояния флага согласия на обработку персональных данных. Данный механизм предотвращает несанкционированную отправку информации на сервер, если пользователь не подтвердил свое ознакомление с политикой конфиденциальности. В случае нарушения условий выводится системное уведомление, блокирующее дальнейшие действия. Таким образом, JavaScript обеспечивает не только визуальный комфорт, но и соблюдение правовых норм взаимодействия с веб-ресурсом. Дополнительная логика в компоненте `Stages.jsx` управляет процессом взаимодействия с документацией, обеспечивая мгновенную реакцию интерфейса на попытку ознакомления с договорами подряда, что повышает общую информативность и прозрачность деятельности компании «Евклид».

3 Валидация кода и продвижение сайта

3.1 Валидация кода сайта

Процесс технического контроля качества программного продукта «Евклид» включал в себя многоуровневую проверку всех составляющих верстки и программной логики. На этапе написания разметки использовались автоматизированные средства валидации, подтверждающие соответствие итогового HTML-кода современным стандартам Консорциума Всемирной паутины (W3C). Это гарантирует, что структура страницы будет корректно интерпретироваться всеми актуальными браузерными движками, обеспечивая кроссбраузерность и стабильность отображения контента. Особое внимание было уделено валидации каскадных таблиц стилей, где проверялось отсутствие синтаксических ошибок и корректность использования экспериментальных свойств с соответствующими префиксами.

Для поддержания высокого качества JavaScript-кода в проект была интегрирована система статического анализа ESLint. Данный инструмент позволил на ранних этапах разработки выявлять потенциально опасные конструкции и следовать единому стилю кодирования, что критично для командной разработки и долгосрочной поддержки проекта. Дополнительная обработка файлов через форматтер Prettier обеспечила идеальную читаемость исходного кода, исключая расхождения в форматировании. Техническая валидация также затронула вопросы безопасности данных: программный механизм проверки обязательного согласия пользователя на обработку персональных данных перед отправкой формы представлен на рисунке 12. Это позволило добиться высокой оценки надежности системы в тестах Lighthouse и обеспечить безопасность пользовательского взаимодействия.

```
const Footer = () => {
  const [isChecked, setIsChecked] = useState(false);

  const handleSubmit = (e) => {
    if (!isChecked) {
      e.preventDefault();
      alert("Пожалуйста, подтвердите согласие на обработку данных");
    }
  };
};
```

Рисунок 12 — Программная валидация формы обратной связи
(src/components/Footer.jsx, строки 6-11)

3.2 Способы продвижения сайта

Эффективность корпоративного веб-ресурса напрямую зависит от его видимости в поисковых системах, что требует реализации комплексного набора мер по продвижению. Основным вектором здесь является поисковая оптимизация (SEO), которая начинается с архитектурного проектирования сайта. Теоретическая база продвижения строится на обеспечении максимальной релевантности контента запросам пользователей, что достигается через проработку семантического ядра. Важным аспектом является техническое совершенство сайта: поисковые роботы отдают предпочтение ресурсам с высокой скоростью загрузки, адаптивной версткой для мобильных устройств и отсутствием критических программных ошибок. Также существенную роль играет внешнее цитирование и авторитетность домена, которые формируются через качественные обратные ссылки.

3.3 Используемые приемы продвижения сайта

В ходе реализации проекта для компании «Евклид» были применены конкретные практические приемы, направленные на повышение позиций сайта в поисковой выдаче. Прежде всего, в заголовочной части index.html были прописаны уникальные мета-теги title and description, содержащие высокочастотные ключевые слова, специфичные для строительного рынка («строительные решения», «проектирование объектов», «Евклид»). Программная реализация мета-тегов и протокола Open Graph в коде страницы

представлена на рисунке 13. Это позволяет поисковым системам формировать привлекательные сниппеты, повышая кликабельность ссылки в результатах поиска. Структура контента выстроена с соблюдением строгой иерархии заголовков h1-h3, что облегчает индексацию наиболее важных смысловых блоков.

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <meta name="description" content="Евклид – строительные решения любой сложности" />
  <meta name="keywords" content="Евклид, строительство, ремонт, строительные работы" />
  <meta name="author" content="Евклид" />
  <meta property="og:title" content="Евклид – Строительные решения" />
  <meta property="og:description" content="Профессиональные строительные решения" />
  <meta property="og:type" content="website" />
  <title>Евклид – Строительные решения любой сложности</title>
</head>
```

Рисунок 13 — Оптимизация мета-тегов и протокола Open Graph (index.html, строки 5-13)

Для улучшения социального взаимодействия были интегрированы теги протокола Open Graph. Это обеспечивает корректное и эстетичное превью сайта при пересылке ссылок в популярных мессенджерах и социальных сетях, что способствует органическому притоку трафика. Настройка атрибутов alt для всех графических элементов проекта не только повышает доступность интерфейса для пользователей с ограниченными возможностями, но и позволяет изображениям участвовать в тематическом поиске по картинкам. Дополнительно была проведена работа по оптимизации путей к статическим ресурсам и минимизации размера шрифтовых файлов, что положительно сказывается на поведенческих факторах, так как пользователи меньше времени проводят в ожидании загрузки контента. Все вышеперечисленные меры в совокупности формируют надежный фундамент для дальнейшего маркетингового развития бренда «Евклид» в сети Интернет.

ЗАКЛЮЧЕНИЕ

В ходе выполнения данной курсовой работы был успешно спроектирован и реализован современный корпоративный веб-сайт строительной компании «Евклид». Использование библиотеки React в сочетании с инструментами автоматизации сборки позволило создать высокопроизводительное одностраничное приложение, отвечающее актуальным требованиям к качеству программных продуктов. Разработанный интерфейс обладает полной адаптивностью, обеспечивая корректное отображение контента на широком спектре абонентских устройств, от настольных компьютеров до мобильных телефонов.

Реализованные на языке JavaScript сценарии поведения элементов, такие как система умного поиска, интеллектуальная кнопка возврата на страницу и механизмы плавной навигации, значительно повышают уровень юзабилити ресурса. Проведенный комплекс мер по поисковой оптимизации и технической валидации кода гарантирует стабильную работу сайта и его доступность для целевой аудитории. Полученный практический опыт разработки SPA-приложения подтверждает эффективность выбранного технологического стека для решения задач автоматизации бизнеса и репрезентации бренда в цифровой среде. Работа полностью соответствует методическим требованиям и может быть использована в качестве основы для разработки более функциональных информационных систем в строительной отрасли.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГУАП, документация для учебного процесса. – URL: <https://guap.ru/regdocs/docs/uch> (дата обращения 08.03.2026)
2. TypeScript подробная англоязычная документация. – URL: <https://www.typescriptlang.org/docs/> (дата обращения 22.02.2026)
3. Фреймворк для JavaScript React документация. – URL: <https://ru.legacy.reactjs.org/docs/getting-started.html> (дата обращения 23.02.2026)
4. Гергель, В.П. Современные языки и технологии параллельного программирования: Учебник / В.П. Гергель. — М.: МГУ, 2016. — 408 с
5. Программирование для начинающих сост.: А.Н. Васильев, изд. «Бомбора», 2024г
6. Официальное руководство по Git и GitHub – URL: <https://git-scm.com/doc> (дата обращения 27.02.2026)
7. Гуджон, Д. Основы веб-разработки. — М.: БХВ-Петербург, 2021. — 400 с
8. Кастро, Э. HTML и CSS: Разработка и дизайн веб-сайтов. — М.: Питер, 2021. — 480 с

ПРИЛОЖЕНИЕ

Исходный код программы

```
// =====  
// File: src/App.jsx  
// =====  
  
import React from 'react';  
import Header from './components/Header';  
import Hero from './components/Hero';  
import About from './components/About';  
import Stages from './components/Stages';  
import FAQ from './components/FAQ';  
import Footer from './components/Footer';  
import ScrollToTop from './components/ScrollToTop';  
  
// Корневой компонент приложения, объединяющий все секции single-page  
application  
function App() {  
  return (  
    <>  
      <Header />  
      <main className="main">  
        <Hero />  
        <About />  
        <Stages />  
        <FAQ />  
      </main>  
      <Footer />  
      <ScrollToTop />  
    </>  
  );  
}
```

```

    </>
  );
}

export default App;

// =====
// File: src/components/Header.jsx
// =====

import React, { useState } from 'react';

// Компонент шапки сайта с логикой плавного скроллинга и поисковой
// системой
const Header = () => {
  const [isSearchOpen, setIsSearchOpen] = useState(false);
  const [searchQuery, setSearchQuery] = useState("");

  // Функция для реализации плавного перемещения к выбранному разделу
  const handleScroll = (e, targetId) => {
    e.preventDefault();
    const elem = document.getElementById(targetId);
    if (elem) {
      elem.scrollIntoView({ behavior: 'smooth' });
    }
  };

  // Обработчик формы поиска с картой ключевых слов для навигации
  const handleSearch = (e) => {
    e.preventDefault();

```

```

const query = searchQuery.toLowerCase().trim();

// Карта соответствия пользовательских запросов ID разделов
const keywords = {
  'о нас': 'about',
  'о компании': 'about',
  'кто мы': 'about',
  'этапы': 'stages',
  'как мы работаем': 'stages',
  'вопросы': 'faq',
  'ответы': 'faq',
  'чаво': 'faq',
  'заявка': 'footer',
  'оставить заявку': 'footer',
  'контакты': 'footer',
  'связь': 'footer'
};

const targetId = keywords[query];
if (targetId) {
  const elem = document.getElementById(targetId);
  if (elem) {
    elem.scrollIntoView({ behavior: 'smooth' });
    setIsSearchOpen(false);
    setSearchQuery("");
    return;
  }
}

// Алгоритм поиска по частичному совпадению (fuzzy match)

```

```

const foundKeyword = Object.keys(keywords).find(k => query.includes(k) ||
k.includes(query));
if (foundKeyword) {
  const elem = document.getElementById(keywords[foundKeyword]);
  if (elem) {
    elem.scrollIntoView({ behavior: 'smooth' });
    setIsSearchOpen(false);
    setSearchQuery("");
  }
}
};

return (
  <header className="header">
    <div className="container header__shape">
      <a className="header__image" href="#">
        
      </a>
      <nav className={`header__nav ${isSearchOpen ? 'header__nav--hidden' :
"}}`}>
        <ul className="header__nav-list list-reset">
          <li className="header__nav-list-item">
            <a className="header__text" href="#about" onClick={(e) =>
handleScroll(e, 'about')}>О нас</a>
          </li>
          <li className="header__nav-list-item">
            <a className="header__text" href="#stages" onClick={(e) =>
handleScroll(e, 'stages')}>Этапы</a>
          </li>
          <li className="header__nav-list-item">

```

```

    <a className="header__text" href="#faq" onClick={(e) =>
handleScroll(e, 'faq')}>Вопросы</a>
  </li>
  <li className="header__nav-list-item">
    <a className="header__text" href="#footer" onClick={(e) =>
handleScroll(e, 'footer')}>Оставить заявку</a>
  </li>
</ul>
</nav>

```

```

  {/* Контейнер расширяемой строки поиска с анимацией */}
  <div className={`header__search-container ${isSearchOpen ?
'header__search-container--open' : ''}`>
    <form className="header__search-form" onSubmit={handleSearch}>
      <input
        className="header__search-input"
        type="text"
        placeholder="Что будем искать?"
        value={searchQuery}
        onChange={(e) => setSearchQuery(e.target.value)}
      />
      <button className="header__search-submit btn-reset" type="submit">
        <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
xmlns="http://www.w3.org/2000/svg">
          <rect width="7.56126" height="1.89031" transform="matrix(0.713342
0.700816 -0.713342 0.700816 18.6064 17.3762)" fill="#FF9900"/>
          <path d="M20.9303 10.7726C20.9303 16.1533 16.4855 20.5452
10.9652 20.5452C5.44478 20.5452 1 16.1533 1 10.7726C1 5.39198 5.44478 1
10.9652 1C16.4855 1 20.9303 5.39198 20.9303 10.7726Z" stroke="#FF9900"
strokeWidth="2"/>

```

```

    </svg>
  </button>
</form>
<button
  className="header__search-close btn-reset"
  onClick={() => setIsSearchOpen(false)}
>
  <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
xmlns="http://www.w3.org/2000/svg">
    <rect x="8.93542" y="7.60205" width="11" height="1.89031"
transform="rotate(45 8.93542 7.60205)" fill="#666666"/>
    <rect x="7.60156" y="15.3809" width="11" height="1.89031"
transform="rotate(-45 7.60156 15.3809)" fill="#666666"/>
  </svg>
</button>
</div>

<button
  aria-label="Кнопка найти"
  className={`header__button btn-reset ${isSearchOpen ? 'header__button--
hidden' : ''}`}
  onClick={() => setIsSearchOpen(true)}
>
  <svg className="header__svg" width="24" height="24" viewBox="0 0 24
24" fill="none" xmlns="http://www.w3.org/2000/svg">
    <rect width="7.56126" height="1.89031" transform="matrix(0.713342
0.700816 -0.713342 0.700816 18.6064 17.3762)" fill="currentColor"/>
    <path d="M20.9303 10.7726C20.9303 16.1533 16.4855 20.5452 10.9652
20.5452C5.44478 20.5452 1 16.1533 1 10.7726C1 5.39198 5.44478 1 10.9652

```

```
1C16.4855 1 20.9303 5.39198 20.9303 10.7726Z" stroke="currentColor"
strokeWidth="2"/>
```

```
    </svg>
```

```
  </button>
```

```
</div>
```

```
</header>
```

```
);
```

```
};
```

```
export default Header;
```

```
// =====
```

```
// File: src/components/Hero.jsx
```

```
// =====
```

```
import React from 'react';
```

```
// Компонент баннера (Hero section) с призывом к действию
```

```
const Hero = () => {
```

```
  const handleScroll = (e) => {
```

```
    e.preventDefault();
```

```
    const elem = document.getElementById('footer');
```

```
    if (elem) {
```

```
      elem.scrollIntoView({ behavior: 'smooth' });
```

```
    }
```

```
  };
```

```
  return (
```

```
    <section className="order">
```

```
      <div className="container">
```

```

    <div className="order__banner">
      <h1 className="order__title">Строительные решения любой
сложности</h1>
      <p className="order__paragraph">Комплексный подход к
строительству: от идеи и проектирования до сдачи объекта
«под ключ»</p>
      <button
        aria-label="Кнопка заказать расчёт"
        className="button order__button"
        onClick={handleScroll}
      >
        Заказать расчёт
      </button>
    </div>
  </div>
</section>
);
};

```

```
export default Hero;
```

```

// =====
// File: src/components/Stages.jsx
// =====

```

```
import React from 'react';
```

```

// Секция этапов работ с демонстрацией возможности скачивания
документации
const Stages = () => {

```



```
// =====
// File: src/components/Footer.jsx
// =====

import React, { useState } from 'react';

// Подвал сайта с формой обратной связи и механизмом валидации согласия
// на обработку данных
const Footer = () => {
  const [isChecked, setIsChecked] = useState(false);

  // Валидация перед отправкой данных на сервер
  const handleSubmit = (e) => {
    if (!isChecked) {
      e.preventDefault();
      alert("Пожалуйста, подтвердите согласие на обработку данных");
    }
  };

  return (
    <footer className="footer" id="footer">
      <div className="container footer__shape">
        <div className="footer__card">
          <div className="footer__card-item footer__nav-shape">
            <a className="footer__logo" href="#">
              
            </a>
            <nav className="footer__nav">
              <ul className="footer__nav-list list-reset">
```

```

        <li className="footer__nav-list-item"><a className="footer__text"
href="#">О нас</a></li>

        <li className="footer__nav-list-item"><a className="footer__text"
href="#">Проекты</a></li>

        <li className="footer__nav-list-item"><a className="footer__text"
href="#">Отзывы</a></li>

    </ul>

</nav>

</div>

<div className="footer__card-item footer__form-shape">
    <h2 className="footer__title-two">Оставить заявку</h2>
    <form action="https://jsonplaceholder.typicode.com/posts"
method="POST" onSubmit={handleSubmit}>
        <div className="footer__form-input">
            <input className="footer__input" type="text" name="name"
id="name" placeholder="Фамилия, имя и отчество*" required />
            <input className="footer__input" type="email" name="email"
id="email" placeholder="Email*" required />
            <textarea className="footer__input-area" name="text" id="text"
placeholder="Сообщение"></textarea>
            <div className="footer__form-button">
                <button aria-label="Кнопка отправить данные"
className="footer__submit button" type="submit">Отправить данные</button>
            <div className="footer__checkbox">
                <input
                    className="footer__checkbox-input"
                    type="checkbox"
                    id="checkbox"
                    name="checkbox"
                    checked={isChecked}

```

```

        onChange={(e) => setIsChecked(e.target.checked)}
      />
      <label className="footer__checkbox-label"
htmlFor="checkbox">Согласен на обработку данных</label>
    </div>
  </div>
</div>
</form>
</div>
<div className="footer__card-item footer__card-shape">
  <p className="footer__paragraph paragraph-text">Компания «Евклид»
— это надежный партнер в сфере капитального строительства и
реконструкции. Мы предлагаем инновационные решения, которые позволяют
оптимизировать затраты без потери качества.</p>
</div>
</div>
</div>
</footer>
);
};

export default Footer;

// =====
// File: src/css/style.css
// =====

/* Глобальные настройки шрифтов и типографики */
@font-face {
  font-family: "Montserrat";

```

```

src: local("Montserrat-Regular"),
      url("../fonts/Montserrat/Montserrat-Regular.woff2") format("woff2");
font-weight: 400;
font-style: normal;
font-display: swap;
}

/* Определение системных переменных для цветовой палитры и констант
дизайна */
:root {
  --light-orange: #FFB84D;
  --orange: #FF9900;
  --dark-orange: #E1670E;
  --white: #fff;
}

/* Очистка базовых стилей и настройка бокс-модели */
html {
  box-sizing: border-box;
}

*, *:before, *:after {
  box-sizing: inherit;
}

body {
  font-family: "Montserrat", Arial, sans-serif;
  background-color: var(--white);
  margin: 0;
}

```

```
/* Центральный контейнер для ограничения ширины контента и центровки */  
.container {  
    margin: 0 auto;  
    max-width: 1920px;  
    padding: 0 75px;  
}
```

```
/* Универсальные стили для кнопок с состояниями hover/active */  
.button {  
    color: var(--white);  
    background-color: var(--orange);  
    border: none;  
    border-radius: 16px;  
    padding: 15px 30px;  
    font-weight: 700;  
    cursor: pointer;  
    transition: background-color 0.3s ease;  
}
```

```
.button:hover {  
    background-color: var(--light-orange);  
}
```

```
.button:active {  
    background-color: var(--dark-orange);  
}
```

```
/* Настройки шапки сайта с использованием Flexbox */  
.header {
```

```
margin-bottom: 50px;
padding: 14px 0;
box-shadow: 0px 5px 10px rgba(0, 0, 0, 0.1);
background-color: var(--white);
}
```

```
.header__shape {
  display: flex;
  align-items: center;
  justify-content: space-between;
}
```